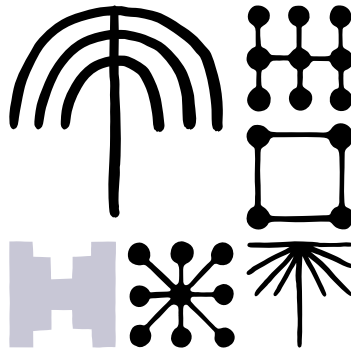


通过 MCP 执行代码：构建更高效的 Agent

Code execution with MCP: Building more efficient agents

Anthropic · 2025-11-04 · 原文 / Source

完整译文与英文逐段对照。依据你提供的网页存档翻译，保留原文的观点、例子与限定条件。图表（含图注）、代码及其中的文字维持原文；网页导航、订阅表单和站点页脚不属于文章正文。中文为学习用非官方译文。



直接调用工具时，每个工具定义和返回结果都会占用上下文。让 Agent 编写代码来调用工具，则更有利于扩展规模。下面介绍这种方式如何与 MCP 配合。

模型上下文协议（MCP）是一项将 AI Agent 连接到外部系统的开放标准。传统上，每一种 Agent 与工具或数据的配对都需要定制集成，由此产生的碎片化和重复劳动，使真正互联的系统难以扩展。MCP 提供了一种通用协议：开发者只需在 Agent 中实现一次 MCP，就能接入整个集成生态。

自 2024 年 11 月发布以来，MCP 迅速得到采用：社区已构建了数千个 MCP 服务器，所有主流编程语言都有可用的 SDK，行业也已将 MCP 作为连接 Agent、工具和数据的事实标准。

如今，开发者经常构建能够通过数十个 MCP 服务器访问数百乃至数千个工具的 Agent。然而，随着连接的工​​具增多，一开始就加载所有工具定义，以及让中间结果经过上下文窗口，都会拖慢 Agent 的运行速度并增加成本。

本文将探讨如何借助代码执行，让 Agent 更高效地与 MCP 服务器交互，在使用更少 token 的同时处理更多工具。

工具消耗过多 token，会降低 Agent 的效率

随着 MCP 使用规模扩大，两种常见模式会增加 Agent 的成本和时延：

1. 工具定义使上下文窗口不堪重负；
2. 工具中间结果消耗额外的 token。

1. 工具定义使上下文窗口不堪重负

大多数 MCP 客户端会预先将全部工具定义直接加载进上下文，通过直接工具调用的语法将其提供给模型。这些工具定义可能如下所示：

```
gdrive.getDocument
  Description: Retrieves a document from Google Drive
  Parameters:
    documentId (required, string): The ID of the document to retrieve
    fields (optional, string): Specific fields to return
  Returns: Document object with title, body content, metadata, permissions, etc.
```

```
salesforce.updateRecord
  Description: Updates a record in Salesforce
  Parameters:
    objectType (required, string): Type of Salesforce object (Lead, Contact,
    Account, etc.)
    recordId (required, string): The ID of the record to update
    data (required, object): Fields to update with their new values
  Returns: Updated record object with confirmation
```

工具描述会占用更多上下文窗口空间，增加响应时间与成本。当 Agent 连接了数千个工具时，它甚至还没读到用户请求，就需要先处理数十万个 token。

2. 工具中间结果消耗额外的 token

大多数 MCP 客户端允许模型直接调用 MCP 工具。例如，你可能要求 Agent：“从 Google Drive 下载我的会议转录，并将它附加到 Salesforce 的销售线索上。”

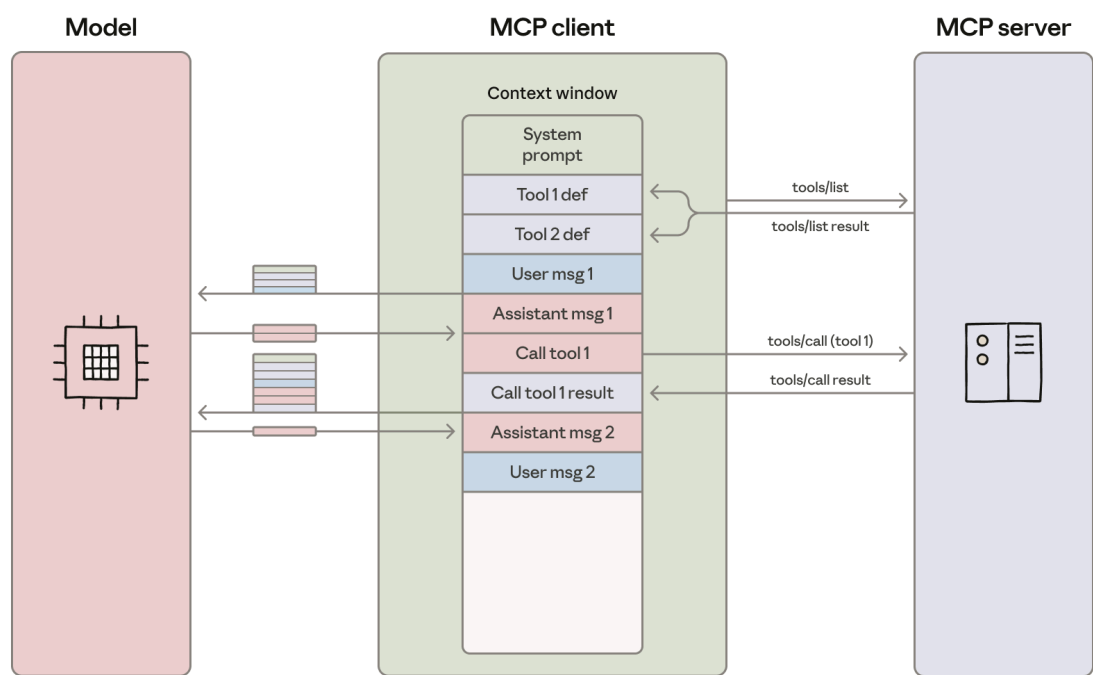
模型会进行类似下面的调用：

```
TOOL CALL: gdrive.getDocument(documentId: "abc123")
  → returns "Discussed Q4 goals...\n[full transcript text]"
    (loaded into model context)

TOOL CALL: salesforce.updateRecord(
  objectType: "SalesMeeting",
  recordId: "00Q5f000001abcXYZ",
  data: { "Notes": "Discussed Q4 goals...\n[full transcript text written out]"
}
)
(model needs to write entire transcript into context again)
```

每个中间结果都必须经过模型。在这个例子中，完整的通话转录会流经模型两次。对于一场两小时的销售会议，这可能意味着需要额外处理 50,000 个 token。更大的文档甚至可能超出上下文窗口限制，导致工作流程中断。

面对大型文档或复杂数据结构时，模型在不同工具调用之间复制数据，也更容易出错。



The MCP client loads tool definitions into the model's context window and orchestrates a message loop where each tool call and result passes through the model between operations.

通过代码执行使用 MCP，提高上下文利用效率

随着代码执行环境在 Agent 中越来越常见，一种解决方案是将 MCP 服务器呈现为代码 API，而非直接工具调用。这样，Agent 就可以编写代码与 MCP 服务器交互。这个方法同时解决了上述两个挑战：Agent 可以只加载自己需要的工具，并在执行环境中处理数据，然后再把结果返回给模型。

实现方式有很多。一种方法是为已连接的 MCP 服务器中的所有可用工具生成文件树。下面是一个 TypeScript 实现：

```
servers
├── google-drive
│   ├── getDocument.ts
│   ├── ... (other tools)
│   └── index.ts
├── salesforce
│   ├── updateRecord.ts
│   ├── ... (other tools)
│   └── index.ts
└── ... (other servers)
```

然后，每个工具对应一个文件，大致如下：

```
// ./servers/google-drive/getDocument.ts
import { callMCPTool } from "../../client.js";

interface GetDocumentInput {
  documentId: string;
}

interface GetDocumentResponse {
  content: string;
}

/* Read a document from Google Drive */
export async function getDocument(input: GetDocumentInput): Promise<GetDocumentResponse> {
  return callMCPTool<GetDocumentResponse>('google_drive__get_document', input);
}
```

上面将数据从 Google Drive 传到 Salesforce 的例子，就变成了这样的代码：

```
// Read transcript from Google Docs and add to Salesforce prospect
import * as gdrive from './servers/google-drive';
import * as salesforce from './servers/salesforce';

const transcript = (await gdrive.getDocument({ documentId: 'abc123' })).content;
await salesforce.updateRecord({
  objectType: 'SalesMeeting',
  recordId: '00Q5f000001abcXYZ',
  data: { Notes: transcript }
});
```

Agent 通过探索文件系统来发现工具：列出 `./servers/` 目录，找到可用服务器，例如 `google-drive` 和 `salesforce`；再读取所需的具体工具文件，例如 `getDocument.ts` 和 `updateRecord.ts`，了解各个工具的接口。这样，Agent 只需加载当前任务所需的定义。这将 token 用量从 150,000 降到了 2,000，节省了 98.7% 的时间与成本。

Cloudflare 也发布了类似发现，将通过代码执行使用 MCP 称为“Code Mode”。核心观点是一样的：大语言模型擅长编写代码，开发者应该利用这一优势，构建能更高效地与 MCP 服务器交互的 Agent。

通过代码执行使用 MCP 的优势

通过代码执行使用 MCP，可以按需加载工具，在数据到达模型前先行过滤，并一次执行复杂逻辑，从而让 Agent 更高效地利用上下文。这种方法在安全性和状态管理方面也有优势。

渐进式披露

模型很擅长浏览文件系统。将工具以代码形式放在文件系统中，可以让模型按需读取工具定义，而不必一开始就全部读入。

另一种方式是在服务器中添加一个 `search_tools` 工具，用来查找相关定义。例如，使用上文假设的 Salesforce 服务器时，Agent 可以搜索“salesforce”，然后只加载当前任务所需的工具。还可以为 `search_tools` 工具加入一个详细程度参数，让 Agent 选择需要的信息层次，例如仅名称、名称加描述，或包含 schema 的完整定义。这也有助于节约上下文，并高效查找工具。

节省上下文的工具结果

处理大型数据集时，Agent 可以在代码中先过滤和转换结果，再将其返回。以获取一张 10,000 行的电子表格为例：

```
// Without code execution - all rows flow through context
TOOL CALL: gdrive.getSheet(sheetId: 'abc123')
    → returns 10,000 rows in context to filter manually

// With code execution - filter in the execution environment
const allRows = await gdrive.getSheet({ sheetId: 'abc123' });
const pendingOrders = allRows.filter(row =>
  row["Status"] === 'pending'
);
console.log(`Found ${pendingOrders.length} pending orders`);
console.log(pendingOrders.slice(0, 5)); // Only log first 5 for review
```

Agent 看到的是 5 行，而非 10,000 行。类似模式也适用于聚合、跨多个数据源的连接，或提取特定字段，而且都不会使上下文窗口膨胀。

更强大、更节省上下文的控制流

循环、条件判断和错误处理，都可以使用熟悉的代码模式来实现，而无需串接单独的工具调用。例如，如果你需要在 Slack 中收到部署通知，Agent 可以编写：

```
let found = false;
while (!found) {
  const messages = await slack.getChannelHistory({ channel: 'C123456' });
  found = messages.some(m => m.text.includes('deployment complete'));
  if (!found) await new Promise(r => setTimeout(r, 5000));
}
console.log('Deployment notification received');
```

这种方法比在 Agent 循环中交替调用 MCP 工具和 `sleep` 命令更高效。

此外，直接写出可执行的条件分支树，还能减少“首 token 时间”带来的延迟：无需等待模型判断 if 语句，Agent 可以让代码执行环境来完成这一步。

保护隐私的操作

Agent 通过代码执行使用 MCP 时，中间结果默认保留在执行环境中。这样，Agent 只会看到你明确记录或返回的内容。这意味着，不希望与模型共享的数据，可以在工作流中流转，而始终不进入模型的上下文。

对于更加敏感的工作负载，Agent 运行框架可以自动用代号替换敏感数据。例如，假设你需要把电子表格中的客户联系方式导入 Salesforce。Agent 会编写：

```
const sheet = await gdrive.getSheet({ sheetId: 'abc123' });
for (const row of sheet.rows) {
  await salesforce.updateRecord({
    objectType: 'Lead',
    recordId: row.salesforceId,
    data: {
      Email: row.email,
      Phone: row.phone,
      Name: row.name
    }
  });
}
console.log(`Updated ${sheet.rows.length} leads`);
```

MCP 客户端会截获数据，在其到达模型之前，将个人身份信息（PII）替换为代号：

```
// What the agent would see, if it logged the sheet.rows:
[
  { salesforceId: '00Q...', email: '[EMAIL_1]', phone: '[PHONE_1]', name: '[NAME_1]' },
  { salesforceId: '00Q...', email: '[EMAIL_2]', phone: '[PHONE_2]', name: '[NAME_2]' },
  ...
]
```

随后，当这些数据通过另一个 MCP 工具调用传递时，MCP 客户端会通过查表，把代号还原成原始数据。真实的电子邮件地址、电话号码和姓名会从 Google Sheets 流向 Salesforce，但不会经过模型。这可以防止 Agent 意外记录或处理敏感数据。你也可以用这种方式定义确定性的安全规则，选择允许数据从哪里流出、流向哪里。

状态持久化与 Skills

带有文件系统访问能力的代码执行，让 Agent 能够在不同操作之间维持状态。Agent 可以将中间结果写入文件，以便恢复工作并跟踪进展：

```
const leads = await salesforce.query({
  query: 'SELECT Id, Email FROM Lead LIMIT 1000'
});
const csvData = leads.map(l => `${l.Id},${l.Email}`).join('\n');
await fs.writeFile('./workspace/leads.csv', csvData);

// Later execution picks up where it left off
const saved = await fs.readFile('./workspace/leads.csv', 'utf-8');
```

Agent 还可以把自己编写的代码持久保存为可复用函数。为某个任务开发出可运行的代码后，它就能保存这个实现，供未来使用：

```
// In ./skills/save-sheet-as-csv.ts
import * as gdrive from './servers/google-drive';
export async function saveSheetAsCsv(sheetId: string) {
  const data = await gdrive.getSheet({ sheetId });
  const csv = data.map(row => row.join(',')).join('\n');
  await fs.writeFile(`./workspace/sheet-${sheetId}.csv`, csv);
  return `./workspace/sheet-${sheetId}.csv`;
}

// Later, in any agent execution:
import { saveSheetAsCsv } from './skills/save-sheet-as-csv';
const csvPath = await saveSheetAsCsv('abc123');
```

这与 Skills 的概念紧密相关。Skills 是包含可复用指令、脚本和资源的文件夹，用于提高模型在专门任务上的表现。为这些已保存的函数添加一个 SKILL.md 文件，就能形成结构化的技能，供模型查阅和使用。随着时间推移，Agent 可以由此积累一套更高层次能力的工具箱，不断演进出让自己最高效工作的支撑结构。

需要注意，代码执行也会带来额外复杂性。运行 Agent 生成的代码，需要一个安全的执行环境，具备适当的沙箱隔离、资源限制和监测能力。这些基础设施要求会增加运维开销和安全方面的考量，而直接工具调用则不需要承担这些负担。代码执行带来的好处，包括降低 token 成本、减少时延和改善工具组合能力，应当与这些实现成本一并权衡。

总结

MCP 为 Agent 连接众多工具和系统提供了基础协议。然而，一旦连接的服务器过多，工具定义和结果就可能消耗过量 token，降低 Agent 效率。

虽然这里的许多问题，如上下文管理、工具组合和状态持久化，看起来很新，但软件工程中已经存在成熟的解决方案。代码执行将这些既有模式应用到 Agent 中，使其能利用熟悉的编程结构，更高效地与 MCP 服务器交互。如果你实现了这种方法，我们欢迎你与 MCP 社区分享发现。

致谢

本文由 Adam Jones 和 Conor Kelly 撰写。感谢 Jeremy Fox、Jerome Swannack、Stuart Ritchie、Molly Vorwerck、Matt Samuels 和 Maggie Vo 对本文草稿提出的反馈。